并行数值计算

课程笔记

作者： 朧千夏

日　期： April 10, 2026

版　本： v1.0

Contents

1 并行计算机体系结构 **1**
1.1 并行计算机的分类 1
1.2 SIMD 型并行机 1
1.3 MIMD 型并行机 1
1.3.1 共享存储 MIMD 并行多处理机 1
1.3.2 分布存储 MIMD 并行多处理机 2
1.3.3 分布共享存储 MIMD 并行多处理机 3

2 并行算法 (1) **4**
2.1 概述 4
2.2 数值并行算法引例 5
2.2.1 例 1. SIMD 算法 5
2.2.2 例 2. 同步 MIMD 算法 6
2.2.3 例 3. 异步 MIMD 算法 6

3 并行算法 (2) **10**
3.1 并行算法的复杂性 10

1 并行计算机体系结构

1.1 并行计算机的分类

并行计算机主要指两台或两台以上的处理机连接起来可以并行操作的机器, 亦可称之为并行多处理机.

按照指令流和数据流的不同, 可以把并行计算机分成下面两类:

1. **单指令流多数据流 (SIMD)**: 各处理机同一时刻执行相同的指令, 处理不同的数据.
2. **多指令流多数据流 (MIMD)**: 各处理机同一时刻执行不同的指令, 且处理不同的数据.

1.2 SIMD 型并行机

SIMD 型并行机主要有一下三类:

1. **向量流水线处理机**: 一般通过时间重叠技术实现并行处理; 把一个功能部件分成几个不同的部分, 每一部分执行部分功能.
2. **阵列处理机**: 由成千上万个功能非常简单的处理机构成, 数据以某种方式流经各台处理机, 由它们进行处理.
3. **并行机**: 通过一台前端机执行指令, 而并行机根据指令对数据进行处理, 如 CM-2、Mas Par MP-1、MP-2.

1.3 MIMD 型并行机

促使 MIMD 结构发展的因素主要有以下两点:

1. 支持更高的并行度 (子程序级、任务级或更高).
2. 多台处理机的性价比远比一台处理机高.

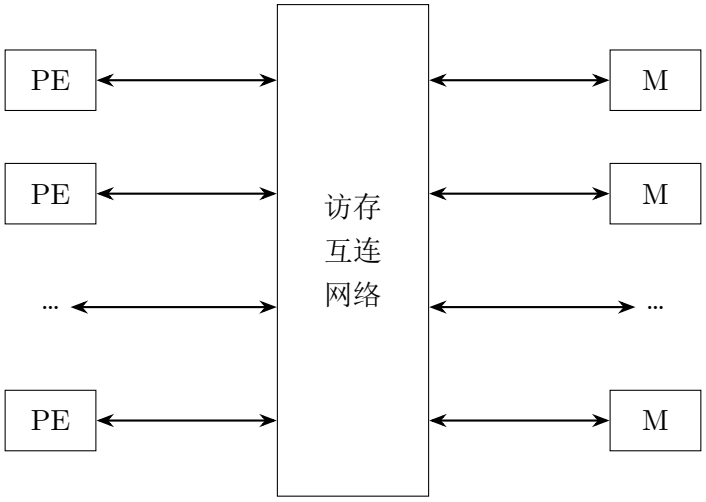
按照存储方式的不同, 我们可以将 MIMD 型并行机可以分成以下三类:

1. **共享存储并行多处理机**.
2. **分布存储并行多处理机**.
3. **分布共享存储并行多处理机**.

1.3.1 共享存储 MIMD 并行多处理机

共享存储 MIMD 并行多处理机是指将多台处理机通过*访存互联网络*共享一个*统一的内存空间*, 并通过该内存空间来实现*处理机间协调*的并行计算系统, 也称为对称多处理机 (SMP, Symmetric Multiple Processors).

共享存储 MIMD 并行多处理机的内存空间也可由多个存储器模块构成, 其结构如图所示:



在该类并行机中, 每台处理机可以执行相同或不同的指令流, 可以直接访问到所有数据, 处理机间通信是借助*共享主存*来实现的.

链接处理机与存储模块之间的网络类型大致可分为以下三种:

1. **总线互连结构**: *分时共享总线*为每台处理机提供了访问内存模块的*均等机会*, 一次只能有一台处理机访问总线; 为了增加处理机台数, 可以采用多总线结构.
2. **Crossbar 互连结构**: 使用*交叉开关*, 在每个可能的处理机/存储器模块之间提供专用通道, 从而防止出现访存竞争冲突. 这种互连配置价格较贵, 处理机数目一般在 4-16 台之间.
3. **多级互连网络结构**: 它是 Crossbar 和总线性价比的折中. 通过*多级互连开关互连网络* (由多级小型交换单元级联构成, 数据通过各级单元按地址逐段选路转发) 连接处理机与存储器模块; 相对于前面两种结构, 具有更好的可扩展性, 但内存访问延迟时间较长.

在共享存储 MIMD 并行机中, 处理机和内存模块分别位于互连网络的两侧, 每台处理机通过互连网络访问内存模块, 且所有处理机的访问方式都是一样的, 机会均等. 同时, 还可以在每台处理机中设置少量的局部 Cache, 以减轻互联网络的负载. 我们称这种共享存储结构为*一致内存访问 (UMA, Uniform Memory Access) 结构*.

UMA 结构的优点在于给每台处理机提供同样的访存带宽、访存机会和访存延迟时间, 处理机与内存模块相对于互连网络呈对称分布, 处理机之间是完全等价的, 由此可以解释为什么我们也称之为 SMP.

共享存储 MIMD 并行多处理机的主要问题是其可扩展性差且不可避免地遇到内存访问瓶颈的问题, 当处理机需要同时访问共享全局变量时, 产生内存竞争现象, 严重影响效率. 其优点是容易进行并行程序设计, 能保持较好的负载平衡, 适合许多中小规模应用问题的计算和事务处理.

1.3.2 分布存储 MIMD 并行多处理机

分布存储 MIMD 并行多处理机中的每台处理机都有自己可直接访问的内存 (局部存储器). 每台处理机称为系统中的一个节点, 节点之间通过*互连网络相互连接*. 每台处理机只能直接访问局部存储器, 对远端存储器的访问必须以*消息传递*的方式, 通过互连网络来完成. 互连网络通常有总线互连结构、环结构、树结构.

> 注: 共享存储 MIMD 并行多处理机的 (访存) 互连网络和分布存储 MIMD 并行多处理机的互连网络在概念上是有所区别的, 在讲到互连网络时注意区分.

分布式存储并行机具有很好的可扩展性, 是实现超大规模科学与工程计算的唯一途径. 如果要求获得较高性能, 必须通过消息传递来进行并行程序设计, 并行程序设计复杂, 不容易被用户接受.

消息传递型分布式存储 MIDI 并行机主要有下面两类:

1. **大规模并行计算机 (MPP)**: 由成百上千的功能相同的处理机通过互联网络连接而成, 如: IBM SP-2, Intel Paragon XP/S, Cray T3D, 国产 YH-3 等.
2. **网路并行机群系统**: 由同构或异构型串行或并行计算机通过快速局域网或广域网相互松散连接而成, 如: 工作站机群、PC 机群等.

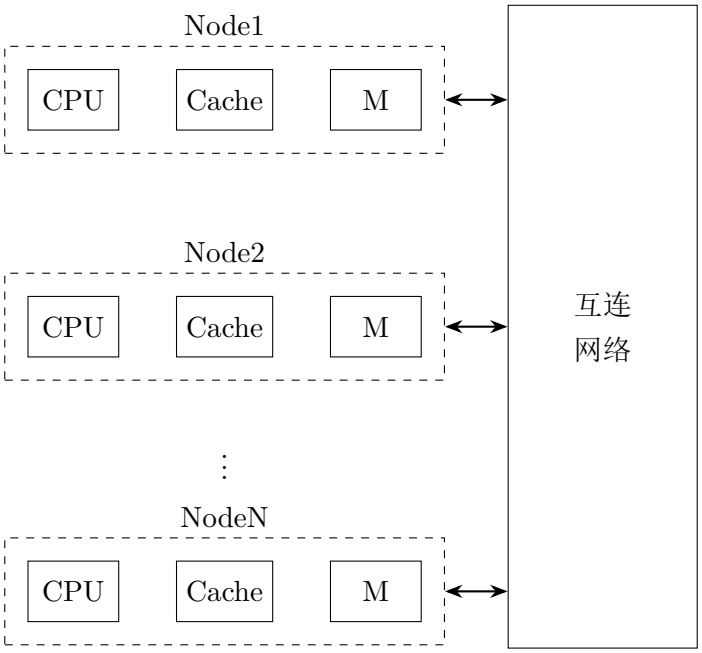
由于工作站机群、PC 机群充分利用各个单位自己的现有资源来组织并行计算, 特别适合于财力不足的科研单位和大学研究所, 近年来得到了快速的发展.

1.3.3 分布共享存储 MIMD 并行多处理机

为了结合 MPP 的可扩展性和 SMP 的并行程序设计的容易性, 提出一种新的结构, 即分布共享存储 (DSM) MIMD 并行计算机.

DSM 结构将 SMP 的内存模块分布到每台处理机, 成为它们的局部内存, 构成一个同一的节点, 节点之间通过互连网络相互连接. 每台处理机可以直接访问它的局部内存, 也可以通过互连网络直接访问别的处理机的局部内存, 只是延迟时间较长.

DSM 提供给用户的是统一的内存空间, 但在物理上类似于 MPP 结构, 是分布内存的, 故称之为分布共享存储结构并行机, 也称之为非一致内存访问 *(NUMA, Non-Uniform Memory Access)* 结构.



DSM 结构具有较好的可扩展性, 可以减轻 SMP 结构的访存瓶颈, 但是又继承了其并行程序设计的容易性.

DSM 的关键技术在于如何保持处理机内部的 Cache 及内存之间的一致性, 不解决这些问题同样会阻碍 DSM 的可扩展性和性能的发挥.

2 并行算法 (1)

2.1 概述

算法是解题方法的精确描述, 它是一组有穷的规则, 这些规则规定了解决某一特定类型问题的一系列运算. 所谓并行算法, 可以简单地认为是在各类并行计算机上求解问题的算法.

为了比较精确地给出并行算法的定义, 我们需要用到计算机科学中的进程 (Process) 的概念. 一个进程是指一段 (串行) 程序连同其数据在 (单) 处理机上的动态执行过程. 简单地说, 进程可以理解为顺序执行的一组操作或一段程序.

> 我们还会遇到和进程密切相关的一个概念: 线程. 在 *Programming Massively Parallel Processors* 一书中这样定义线程: "The sequence of instruction execution activities resulting from this sequential, stepwise execution of an application is referred to as a thread of execution, or simply thread." 可以看到, 线程是执行的最小单位, 线程不独立拥有内存地址空间, 它共享进程的资源. 一个进程可以分为多个线程, 进程是资源分配的最小单位.

接下来给出并行算法的精确定义: 并行算法是一些可同时执行的诸进程的集合, 这些进程相互作用和协调动作, 从而达到对给定问题的求解.

并行算法可以从不同的角度加以分类:

1. **数值**和**非数值**并行算法.
2. **同步**和**异步**并行算法.
3. **SIMD** 和 **MIMD** 并行算法等.

数值并行算法

数值计算是指基于代数关系的一类计算问题, 基本上属于数值分析 (对以数字形式表达的问题求解) 的范畴, 研究数值计算问题的并行算法称为数值并行算法.

非数值并行算法

非数值计算是指基于关系运算的一类计算问题, 基本上属于符号 (如字符、数字图形或别的一些记号) 处理的范畴, 研究非数值计算问题的并行算法称为非数值并行算法.

同步算法

同步算法 (Synchronized Algorithm) 是指某些进程必须等待别的进程的一类并行算法. 因为一个进程的执行依赖于输入数据和系统中断, 所以全部进程均必须同步在一个给定的时钟, 以等待最慢的进程. 一般而言, 运行在 SIMD 计算机上的并行算法为同步并行算法 (Synchronous Paralle Algorithm).

异步算法

异步算法 (Asynchronized Algorithm) 是指进程执行一般不必相互等待的一类并行算法. 异步算法具有一下性质:

1. 有一个可以为所有进程存取的整体变量集合.
2. 当进程的一个阶段完成后, 这个进程首先读某些整体变量, 然后根据这些变量的值及上一阶段刚得到的结果, 该进程修改某些整体变量, 接着启动下阶段或结束它本身.

在异步算法中, 进程之间的通信是通过整体变量可共享数据来实现的. 进程之间没有同步算法那样明显的依赖关系. 异步算法的主要特征是它的进程永不等待输入, 而只根据整体变量的最新信息来继续.

SIMD 算法

面向 SIMD 计算机的并行算法就是 SIMD 算法. SIMD 算法依据 “P 台处理机在同一时刻对各自的数据执行同一指令”, 具有明显的同步性, 属于同步并行算法.

MIMD 算法

面向 MIMD 计算机的并行算法就是 MIMD 算法. MIMD 算法依据 “P 台处理机在同一时刻对各自的数据执行不同指令”. MIMD 算法有同步和异步之区别.

由于使用并行和向量计算机, 我们面临的一个挑战: 在设计算法时, 必须充分利用具体并行计算机的特征.

原有的一些最好的串行算法变成了不能令人满意的算法, 需要重新改造甚至抛弃. 而在串行机上许多远不是最优的 “老” 算法, 由于其并行性而焕发了 “青春”.

传统的数值方法正在经历并行性的改造并在机器上得以检验, 发展新的并行算法的研究工作正在向纵深发展.

2.2 数值并行算法引例

由于本课程为并行数值计算, 我们主要关注于数值并行算法.

数值并行算法的研究内容主要包含:

1. 在传统数值计算方法基础上进行并行性改造和创新.
2. 已经具有明显并行性的传统数值方法和新方法在并行机和向量机上的算法实现和程序设计.

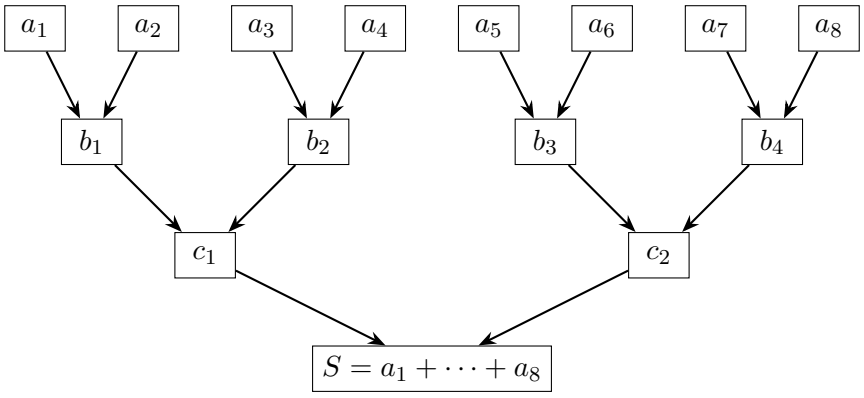
2.2.1 例 1. SIMD 算法

考虑 n 个数 $a_1, \ldots, a_n$ 的求和问题. 其串行算法为:

```
s = a_1
DO i=2,...,n
    s = s + a_i;
```

显然不适合并行计算.

下图给出了 $n = 8$ 时的一种并行算法, 称为**结合扇入算法**, 其可由如下 4 个进程 P_1, P_2, P_3, P_4 组成



P_1	P_2	P_3	P_4
$b_1 \leftarrow a_1 + a_2$ $c_1 \leftarrow b_1 + b_2$ $S \leftarrow c_1 + c_2$	$b_2 \leftarrow a_3 + a_4$	$b_3 \leftarrow a_5 + a_6$ $c_2 \leftarrow b_3 + b_4$	$b_4 \leftarrow a_7 + a_8$

2.2.2 例 2. 同步 MIMD 算法

考虑求一元方程 $f(x) = 0$ 的根的 Newton 迭代法:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}, \quad i = 0, 1, 2, \ldots$$

我们将每次迭代分成三个算法元:

$$\begin{gathered} y_i \leftarrow f(x_i) \\ y_i' \leftarrow f'(x_i) \\ x_{i+1} \leftarrow x_i - y_i/y_i' \text{ 及检验精度} \end{gathered}$$

在一般情况下, 计算 $f(x_i)$ 和 $f'(x_i)$ 所含操作步数是不同的. 因此, 在具有两台速度相同的处理机系统上, 根据公式进行并行计算时, 每次迭代都须同步.

假设计算 $f'(x_i)$ 比计算 $f(x_i)$ 花费更多操作, 下图给出了一种两进程同步 MIMD 算法:

<table>
<tr><th>时间步</th><th>P_1</th><th>P_2</th></tr>
<tr><td>1</td><td>$y_0 = f(x_0)$</td><td rowspan="2">$y_0' = f'(x_0)$</td></tr>
<tr><td>2</td><td></td></tr>
<tr><td>3</td><td>x_1 和检验</td><td></td></tr>
<tr><td>4</td><td>$y_1 = f(x_1)$</td><td rowspan="2">$y_1' = f'(x_1)$</td></tr>
<tr><td>5</td><td></td></tr>
<tr><td>6</td><td>x_2 和检验</td><td></td></tr>
<tr><td>$\vdots$</td><td>$\vdots$</td><td>$\vdots$</td></tr>
</table>

2.2.3 例 3. 异步 MIMD 算法

通过公用变量或公用数据来实现进程间的通信是异步算法的显著特征. 异步算法的进程间没有像同步算法那样有明显的依赖性, 各进程不需要时间以等待信息输入, 不论公用变量中当前信息是什么, 各进程将根据这些信息决定继续执行或终止.

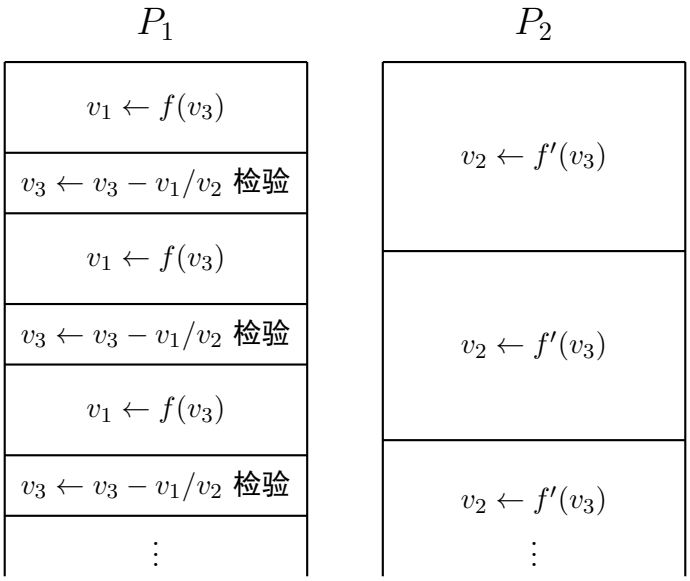
为了保证逻辑上的正确性和进行有意义的工作, 如使必须更新的若干公用变量的更新过程全部完成或解决公用变量的存储冲突等, 必须允许异步算法中出现某些随机的临时封锁.

由于公用变量更新信息有很大的随机性, 直接法不能包含这种量, 所以异步算法基本上是迭代法, 并且与传统迭代法有很大区别.

仍以求 $f(x) = 0$ 的根为例, 引进公用变量 v_1, v_2, v_3 , 分别含 $f(x), f'(x), x$ 的当前值. 仍设计算 $f'(x_i)$ 比计算 $f(x_i)$ 花费更多操作, 下图所示是一个比较合理的两进程异步 MIMD 算法,

或称为异步迭代算法:

P_1 更新 v_1 和 v_3 , 且检验精度, P_2 更新 v_2 .



假定变量的初值 $v_1 = 0$, $v_2 = f'(x_0)$, $v_3 = x_1$, 则依上图计算时, v_3 得出的根的近似值序列如下:

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_0)}$$
$$x_3 = x_2 - \frac{f(x_2)}{f'(x_1)}$$
$$x_4 = x_3 - \frac{f(x_3)}{f'(x_2)}$$

因此, 递推关系形如:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_j)}$$

其中 $j < i$, 当 $i \to \infty$ 时, $j \to \infty$, 但事先无法确定 j 和 i 的关系, 而且此递推形式只适合 "计算 $f'(x_i)$ 比计算 $f(x_i)$ 花费更多操作" 的情形.

这已经不是传统的 Newton 迭代法, 而是一种异步迭代法, 或者可称之为混乱迭代法, 故必须对产生的序列 $\{x_i\}_{i=1}^{\infty}$ 的性质进行深入细致的分析.

按照传统的概念, 数值计算方法和程序设计是两门密切相关的不同学科.

算法是解题方法的精确描述, 它是一组有穷的规则, 这些规则确定了在计算机上求解某一问题的一系列运算. 根据算法可以直接地进行程序设计.

对于串行机而言, 数值方法 (可简称为解法) 与算法的关系比较简单, 解法一旦确定, 有效的串行算法和计算程序也随之产生其可在任何一种串行机上得到实现. 在并行计算系统出现以后, 区别解法和算法这两个概念的重要性增加了, 同样一个解法针对不同类型的并行机可以有着差异很大的算法和程序设计.

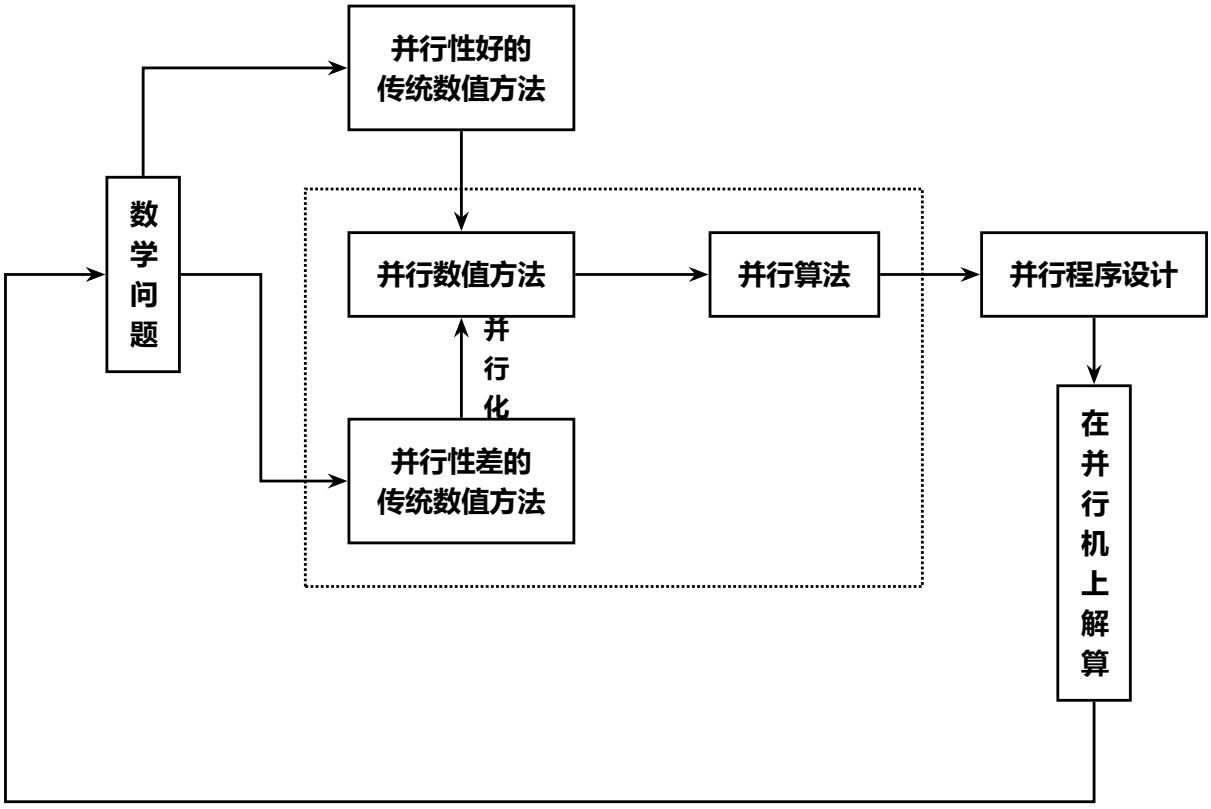
例如, 考虑求解 n 阶稠密线性代数方程组 $Ax = b$ 的 Gauss 消元法. 消元过程如下:

> 设 $a_{ii} \neq 0$, 要求通过矩阵初等变换将 a_{ji} $(i < j \leq n)$ 化为零, 其余元素 a_{kl} 和右端元素 b_k $(i < k, l \leq n)$ 都要进行修正, 变成 a^*_{kl} 和 b^*_k

对于串行情况, Gauss 解法的算法设计比较容易做到, 并且在实际应用上可以找到主元素 Gauss 消元法的标准子程序.

对于并行计算系统, Gauss 解法的算法实现可能有许多途径. 需要考虑到机器的类型和特点, 是并行机还是向量机, 是共享存储类型还是局部存储类型, 并行机的处理器个数有多少, 各处理器之间的负载平衡等问题.

并行算法一经给出, 并行程序即可直接由该算法设计产生. 算法受到数值方法影响和制约, 解法对算法有决定意义. 有效的并行算法一般必须建立在容易并行实现的数值方法基础上, 具有良好并行性的数值方法称为并行数值方法.



虚线所界定的内容构成了一个与并行算法直接相关的相当广泛的研究领域. 为了在并行机上实现给定问题的求解, 需要完成图中所示任务的一个大循环.

对于所要求解的数学问题, 如果传统数值方法已具备良好的和明显的并行性, 则可直接进入并行算法设计; 倘若传统方法没有良好的并行性, 则应对其进行并行性改造和创新.

具备并行性的传统数值方法

用主元素 Gauss 消元法解稠密线性代数方程组、矩阵与向量的乘法运算、求解发展方程的显式差分格式、求解线性代数方程组的 Jacobi 迭代法等

难于并行化的传统数值方法

递推问题、求解抛物型方程的隐式差分格式、椭圆型方程的直接近似解法和若干迭代解法等

这些难于并行化的传统数值方法在大规模科学计算中占有重要地位, 其并行化倍受重视, 近年来有了很大的发展, 提出了一些新技术和新方法. 同时也产生了许多新的理论问题, 如: 并行

迭代法的收敛性、收敛速度的估计、并行化差分格式的相容性、收敛性、稳定性等. 这些问题都需要进一步深入的研究和分析.

需要指出, 并行数值方法的发展并不局限于上述所示框图, 例如不少学者在数学模型一级进行并行化研究, 利用分而治之原则建立了求解数学物理问题的区域分裂法. 甚至为了并行计算的目的, 能够建立全新的计算模型, 如流体力学计算的格子气方法等.

传统数值方法在并行化方面正面临着新的挑战, 也面临着理论上取得突破和新发展的大好机遇. 并行计算新时期的出现孕育着并行数值方法大发展时期的到来, 并行数值方法的新领域目前正在呈现出蓬勃发展的势头.

3 并行算法 (2)

3.1 并行算法的复杂性

一个算法的复杂性 (Complexity) 也叫复杂度, 是指其所含的工作量.

例如, 串行算法的复杂性是指算法的运算量 (即时间步) 与存储量 (存储空间), 其都是求解问题规模的函数. 当问题的规模 n 趋向无穷大时, 就得到算法复杂性的渐近表示. 一般我们关心的是 n 较大时的复杂性, 这时它与渐近复杂性相差不大.

对于在 SIMD 计算机和 MIMD 计算机上的并行算法, 我们研究算法的运行时间和所需处理机的台数与问题规模 n 的变化关系.